Lab - 3

1. SJF - Non-Preemptive

```
# ~~def~~ include <stdio.h>
# define Max 10
struct sjf{
    int pid, bt, at, ct, wt, tat, done;
};
void input (struct sjf f[], int n){
    for (int i=0; i<n; i++){
        f[i].pid = i+1;
        printf("Enter arrival time & burst time of process %d:
                (i+1);
        scanf("%d %d", &f[i].at, &f[i].bt);
        f[i].done = 0;
    }
}
void calculate (struct sjf f[], int n){
    int time = 0, completed = 0;
    for avgtat = 0, avgwt = 0;
    while (completed < n){
        int idx = -1;
        int minbt = 999;
        for (int i=0; i<n; i++){
            if (!f[i].done && f[i].at <= time && f[i].bt < min
                minbt = f[i].bt;
                idx = i;
            }
        }
        if (idx == -1){
            time++;
            continue;
        }
        time += f[idx].bt;
        f[idx].ct = time;
```

```
f[idx].tat = f[idx].ct - f[idx].at;
f[idx].wt = f[idx].tat - f[idx].bt;
f[idx].done = 1;
completed++;
avg Tat += f[idx].tat;
avg wt += f[idx].wt;
}
printf("\nProcess\t AT \t BT \t CT \t TAT \t WT \n");
for(int i=0; i<n; i++){
    printf("P%d \t%d \t%d \t%d \t%d \t%d \n",
  f[i].pid, f[i].at, f[i].bt, f[i].ct, f[i].tat, f[i].wt);
}
printf("\nAverage TAT: %.2f \n", avg Tat/n);
printf("Average WT: %.2f \n", avg wt/n);
}
int main(){
    struct Sj f[MAX];
    int n;
    printf("Enter no of process:");
    scanf("%d", &n);
    input(f,n);
    calculate(f,n);
    return 0;
}
```

O/P.

Enter no of process: 5.

Enter arrival time & burst time of

Process 1: 0
3

Process 0: 2
2

Process 3: 3
5

Process 4: 4
4

Process 5: 6
1

Process	AT	BT	CT	TAT	WT
P_1	0	3	3	3	0
P_2	0	2	5	3	1
P_3	3	5	15	12	7
P_4	4	4	9	5	1
P_5	6	1	10	4	3

P_1	P_2	P_4	P_5	P_3

0 3 5 9 10 15

Average TAT : 8.40

Average WT : 2.40

Q. SJF - Preemptive

```
#include<stdio.h>
#define MAX 10
struct sjf{
    int pid, bt, at, ct, wt, tat, rt; };
void input (struct sjf f[] ,int n){
    for (int i=0; i<n ; i++){
        f[i].pid = i+1;
    printf("Enter arrival time & Burst time :");
    scanf("%d %d", &f[i].at, &f[i].bt);
    f[i].rt = f[i].bt;
    }
}
void calculate(struct sjf f[], int n){
    int time=0, completed=0;
    float avgTat=0, avgWt=0;
    while (completed<n){
        int idx = -1;
        int minRt=1e9;
```

```
for (int i=0; i<n; i++){
    if (f[i].rt > 0 && f[i].at <= time && f[i].rt < minRt){
        minRt = f[i].rt;
        idx = i;
    }
}
if (idx == -1){
    time++;
    continue;
}
f[idx].rt--;
time++;
if (f[idx].rt == 0){
    f[idx].ct = time;
    f[idx].tat = f[idx].ct - f[idx].at;
    f[idx].wt = f[idx].tat - f[idx].bt;
    avgTat += f[idx].tat;
    avgWt += f[idx].wt;
    completed++;
}
}
printf("\nprocess\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0; i<n; i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    f[i].pid, f[i].at, f[i].bt, f[i].ct, f[i].tat, f[i].wt);
}
printf("\nAverage TAT: %.2f\n", avgTat/n);
printf("Average WT: %.2f\n", avgWt/n);
}
int main(){
    struct sjf f[MAX];
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    input(f, n);
    calculate(f, n);
    return 0;
```

O/P:

Enter no. of processes.

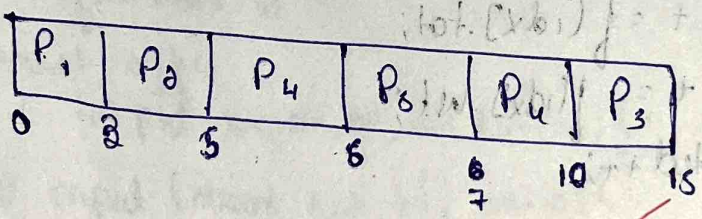
Enter AT & BT:

0	3
2	2
3	5
4	4
6	1

Process	AT	BT	CT	TAT	WT
P_1	0	3	3	3	0
P_2	2	2	5	3	1
P_3	3	5	16	13	7
P_4	4	4	10	6	2
P_5	6	1	7	1	0

Average TAT : 8.0

Average WT : 2.0



3. Priority - Non-Preemptive

```
# include < stdio.h>
# define MAX 10
Struct process{
  int pid, bt, at, pri, ct, wt, tat, done;
};
void input (struct process p[], int n){
   for(int i=0; i<n; i++){
     p[i].pid = i+1;
     printf("Enter AT, bt, Pri:", i+1);
     scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pri);
     p[i].done = 0;
   }
}
void calculate (struct process p[], int n){
   int time=0, completed=0;
   float avgTat=0, avgWt=0;
   while (completed <n){
    int idx = -1, highestPri = 100;
    for(int i=0; i<n; i++){
      if( !p[i].done && p[i].at <= time && p[i].pri < highestPri){
          highest Pri = p[i].pri;
          idx = i;
      }
    }
    if(idx == -1){
      time ++
      continue;
    }
    time += p[idx].bt;
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx]. done = 1;
    completed ++;
```

```
    avgWt+ = p[idx].wt;
}

printf("\nProcess \t AT \t BT \t PR \t CT \t TAT \t WT \n");
for (int i=0; i<n; i++){
    printf("P %d \t %d \t %d \t %d \t %d \t %d \n", p[i].pid, p[i].at,
p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt);
  }
  printf("\nAvg TAT: %.2f \n", avgTat/n);
  printf("Avg WT : %.0f \n", avgWt/n);
}
int main(){
  struct process p[MAX];
  int n;
  printf("Enter number of processes: ");
  scanf("%d", &n);
  input(p, n);
  calculate(p, n);
  return 0;
}
```

O/P.

Enter no of process: 5.

Enter At, Bt, pri :

0	3	5
2	8	3
3	5	2
4	4	4
6	1	1

Process	AT	BT	PR	CT	TAT	WT
P_1	0	3	5	3	3	0
P_2	2	2	3	11	9	7
P_3	3	5	2	8	5	0
P_4	4	4	1	15	11	7
P_5	6	1	1	9	3	2

P_1	P_3	P_5	P_2	P_4

0 3 8 9 11 15

Avg TAT : 6.2
Avg WT : 3.2

4. Priority_Preemptive

```
#include <stdio.h>
#define MAX 10
struct process {
    int pid, bt, at, pr, ct, wt, tat, rt, done;
};

void input(struct process p[], int n){
    for(int i=0; i<n; i++){
        p[i].pid = i+1;
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
        p[i].rt = p[i].bt;
        p[i].done=0;
    }
}
void calculate(struct process p[], int n){
    int time=0, completed=0;
    float avgTat=0, avgWt=0;
    while(completed<n){
        int idx=-1, highestPr=1e9;
        for(int i=0; i<n; i++){
            if (!p[i].done && p[i].at <= time && p[i].pr<
                highestPr){
                highestPr = p[i].pr;
                idx = i;
            }
        }
        if (idx == -1){
            time++;
            continue;
        }
        p[idx].rt --;
        time++;
        if (p[idx].rt == 0){
            p[idx].ct = time;
```

```
        P[idx].wt = P[idx].tat - P[idx].bt;
        P[idx].done = 1;
        completed ++;
        avgTat += P[idx].tat;
        avgWt += P[idx].wt;
    }
}
printf("\nProcess \t AT \t BT \t PR \t CT \t TAT \t WT \n");
for (int i = 0; i<n; i++){
    printf("P %d \t %d \t %d \t %d \t %d \t %d \t %d \n",
    P[i].pid, P[i].at, P[i].bt, P[i].pr, P[i].ct, P[i].tat,
        P[i].wt);

printf("\n Average TAT: %.2f \n", avgTat/n);
printf("Average WT: %.2f \n", avgWt/n);
}

int main(){
    struct process p[MAX];
    int n;
    scanf("%d", &n);
    input(p, n);
    calculate(p, n);
    return 0;
}
```

O/P.

Enter the no of process: 5.

Enter AT, BT, Pri:

0	3	5
2	2	3
3	5	2
4	4	4
6	1	1

Process	AT	BT	PR	CT	TAT	WT.
P_1	0	3	5	15	15	12
P_2	2	2	3	10	8	6
P_3	3	5	2	9	6	1
P_4	4	4	4	14	10	6
P_5	6	1	1	7	1	0.

P_1	P_2	P_3	P_5	P_3	P_2	P_4	P_1

0 2 3 6 7 9 10 14 15

Average TAT : 8.0

Average WT : 5.0.

Rm
9/3/26